CSE326 WEB DEVELOPMENT ACADEMIC PROJECT

CampusBite

Smart Canteen Ordering System

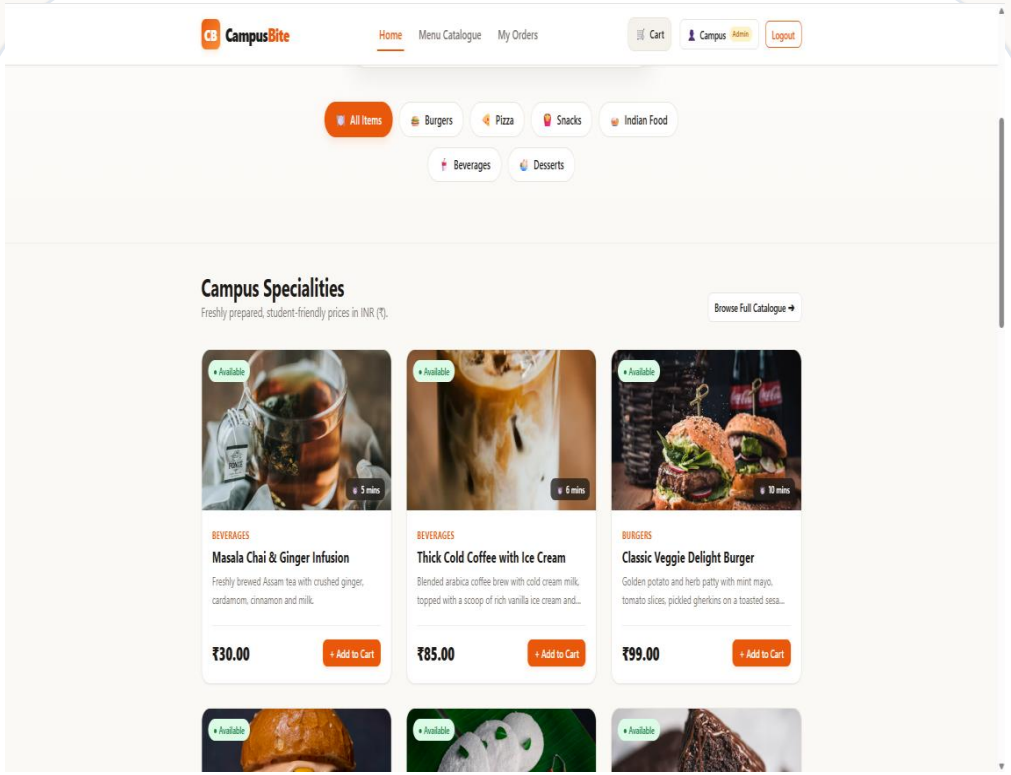
An asynchronous, framework-free web platform engineered to eliminate peak-
campus dining queues through digital ordering, transparent pricing, and real-t
kitchen telemetry.

STUDENT PRESENTER: Pirzada Salik

COURSE & CODE: CSE326 — Web Development

OFFICIAL TEAM: [INSERT OFFICIAL TEAM NAME]

UNIVERSITY: [INSERT UNIVERSITY NAME]



● **LIVE PRODUCTION APP** | Native HTML5 + CSS3 + Vanilla ES6+ & Node.js

CONTEXT & MOTIVATION

The Problem with Traditional Canteen Ordering

Why peak-hour campus dining operations break down during lecture intervals

01 Peak-Hour Counter Congestion

Lecture breaks release hundreds of students simultaneously into the canteen. Physical counters become overwhelmed as students attempt to view physical menus, place orders, and pay all at once.

02 Manual Order Taking & Verbal Friction

Handwritten slips and verbal orders lead to miscommunication, misplaced tokens, and wrong dishes prepared. Cash change shortages further delay transactions at payment registers.

03 Zero Real-Time Status Visibility

After paying, students wait in crowded waiting areas without knowing whether their food is queued, preparing, or ready. This creates anxiety, counter crowding, and missed class time.

04 Administrative Operational Blind Spots

Kitchen staff have no live visibility into incoming queue depth or item demand. Sold-out dishes cannot be dynamically hidden, resulting in frustrating manual cancellations and refunds.

CORE INSIGHT: The bottleneck in campus dining is not kitchen cooking speed — it is the synchronous friction of physical order placement and lack of status visibility.

PRODUCT OVERVIEW

Introducing CampusBite — Smart Digital Ordering

A full-stack, contactless canteen management ecosystem designed for modern campuses

✓ Digital Menu Discovery

Students browse categorized items with live stock badges, prices, and base preparation times.

✓ Transparent Cart & Pricing

Persistent client cart computing exact subtotal, 5% GST, and flat ₹10 convenience fee with zero ambiguity.

✓ Unique Order Token Engine

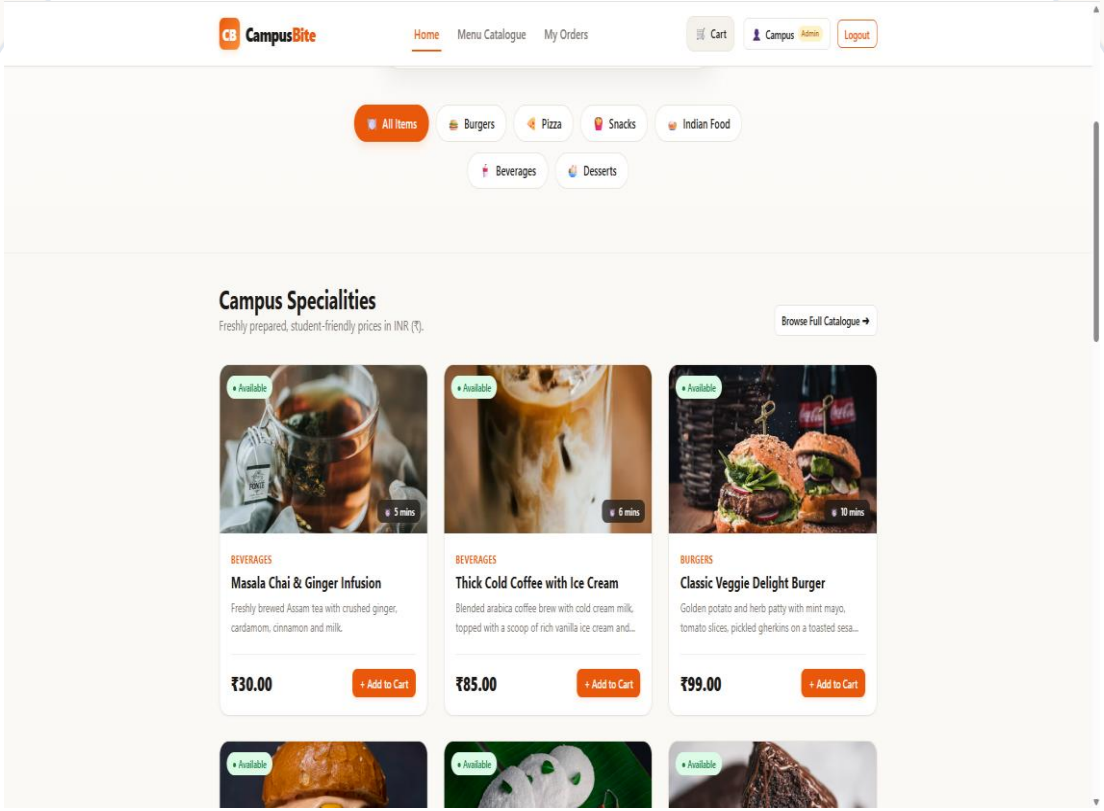
Issues unique tokens (e.g. #CB-521323) upon checkout for quick, error-free counter food pickup.

✓ Live 4-Stage Status Tracking

Students track orders through Placed, Preparing, Ready, and Completed via automated 4.5s API polling.

✓ Centralized Kitchen Command

Single dashboard for canteen operators to adjust item availability, manage queue, and monitor revenue.



Actual Landing Page (/index.html) — Live Food Categories & Real-Time Stock Status

CORE FUNCTIONALITY

One Platform, Two Dedicated Experiences

Tailored, synchronized workflows for students and canteen administrators

STUDENT PORTAL (Self-Service Ordering)

- **User Authentication:** Secure registration & login with persistent session cookies
- **Menu Search & Filters:** 6 food categories (Burgers, Pizza, Snacks, Indian, Drinks, Desserts)
- **Persistent Shopping Cart:** Client-side storage with quantity clamps (1–20 per item)
- **Counter Payment Support:** Select 'Cash at Counter' or 'UPI at Counter' (GPay/PhonePe)
- **Live Kitchen Tracking:** Visual progress stepper with estimated prep time countdown
- **Self-Service Cancellation:** Immediate cancellation allowed exclusively in 'Placed' status
- **Order History Ledger:** Access all past and active orders with digital receipts

CANTEEN ADMIN PORTAL (Kitchen Operations)

- **Command Overview Dashboard:** Live KPIs: Total Orders, Gross Revenue, Active In-Kitchen volume
- **Live Kitchen Order Queue:** Sortable order stream with direct status dropdown controls
- **4-Stage Lifecycle Control:** Enforce transitions: Placed ➔ Preparing ➔ Ready ➔ Completed
- **Counter Payment Settlement:** Collect and verify Cash/UPI; auto-settles to 'Paid' on completion
- **Menu Management CRUD:** Add dishes, update prices, edit prep times, and toggle sold-out state
- **Sales & Demand Analytics:** 7-day revenue trajectory bar charts and top-selling food items
- **Role-Based Route Guards:** Strict authorization blocking unauthorized student access to admin APIs

Architectural Scope: Single centralized canteen administration. No separate multi-vendor panel required.

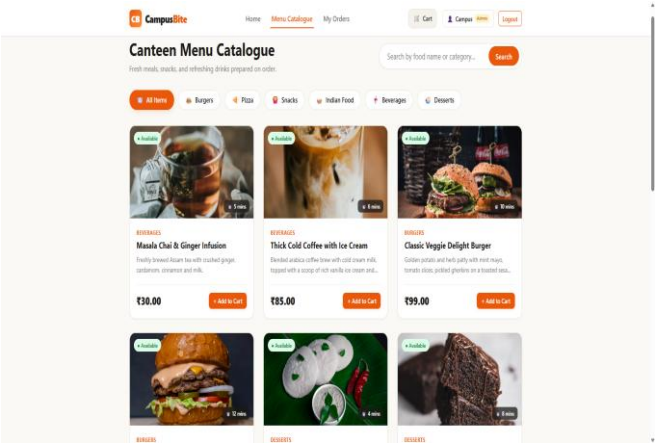
USER EXPERIENCE & FLOW

Student Experience: From Browsing to Ordering

A 3-step streamlined path from campus discovery to order token generation

STEP 1: DISCOVER

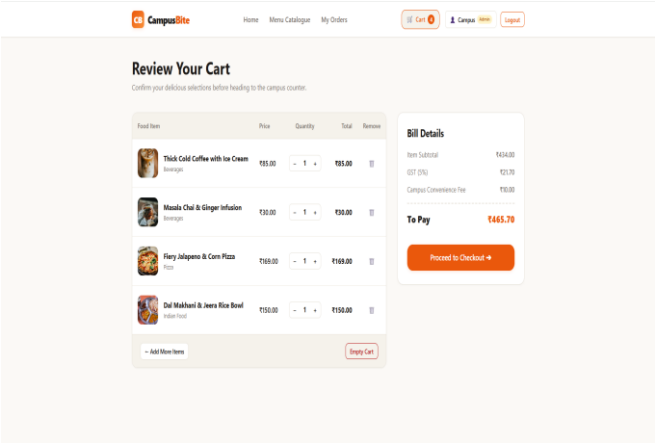
Menu Catalogue



Instant category filtering, search by dish name, live availability indicators, and base preparation times (5m, 10m).

STEP 2: REVIEW

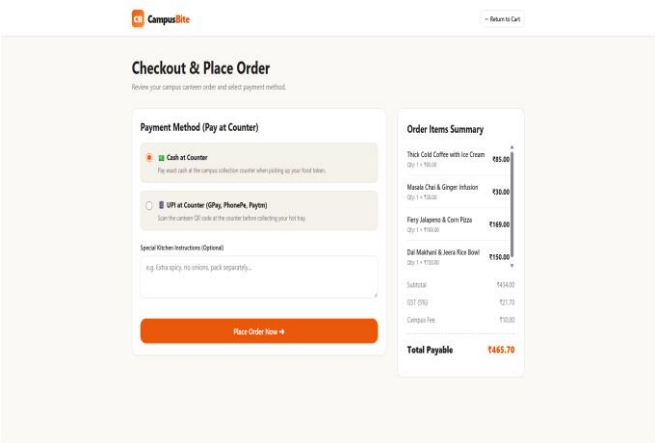
Shopping Cart



Transparent bill calculation: Item subtotal (₹434.00), 5% GST (₹21.70), and flat ₹10.00 campus fee = ₹465.70.

STEP 3: CHECKOUT

Payment & Notes



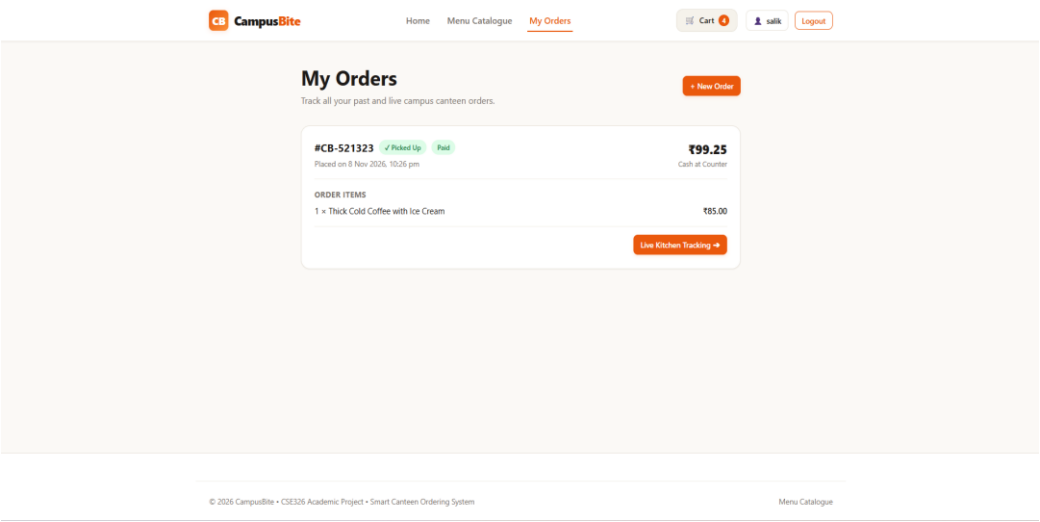
Select 'Cash at Counter' or 'UPI at Counter', add optional kitchen instructions, and submit with duplicate protection.

OPERATIONAL SYNCHRONIZATION

Complete Order Visibility & Kitchen Workflow

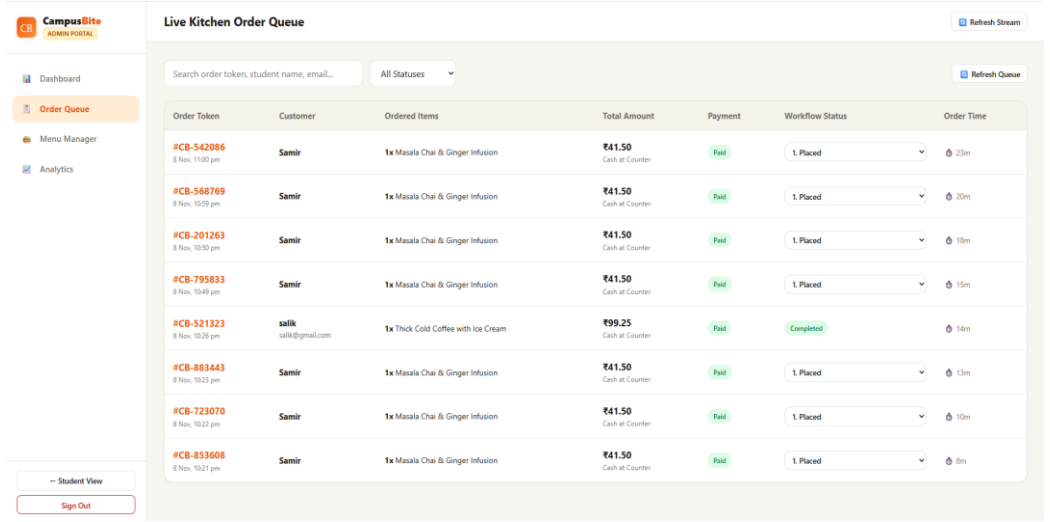
Real-time coordination connecting student status tracking with kitchen fulfillment

STUDENT VIEW: ORDER STATUS & TRACKING



Unique token #CB-521323 with live kitchen tracking trigger & pickup indicators.

ADMIN VIEW: LIVE KITCHEN ORDER QUEUE



Kitchen coordinators advance statuses, inspect items, and track elapsed order times.

ORDER LIFECYCLE STATE MACHINE (SERVER-ENFORCED):

1. Placed ➔ 2. Preparing ➔ 3. Ready ➔ 4. Completed

- Cancellation Guard: Allowed ONLY in 'Placed' state. Once 'Preparing' begins, kitchen lock prevents cancellation.
- Update Protocol: Client polls GET /api/orders/:id every 4.5 seconds for instant reflection without WebSocket overhead.

System Architecture: Three-Tier Foundation

Framework-free engineering meeting CSE326 faculty requirements with production rigor

TIER 1: PRESENTATION

Client Browser

- **Frontend Stack:** Pure HTML5, Semantic CSS3, Vanilla ES6+ JavaScript
- **No Frameworks:** Zero React, Vue, Angular, TypeScript, Tailwind, or Bootstrap
- **DOM & State:** Native Fetch API, DOM manipulation, responsive CSS Grid & Flexbox
- **Persistence:** Session cookies & localStorage for responsive client state

TIER 2: APPLICATION

Node.js + Express 5

- **Server Runtime:** Node.js v24 LTS + Express.js 5.3 REST API engine
- **Security Layers:** Helmet security headers, express-rate-limit protection
- **Clean Structure:** Controllers delegate logic to services (orderService, menuService)
- **Session Auth:** express-session authenticated via connect-mongo & bcryptjs

TIER 3: DATA TIER

MongoDB Atlas Cloud

- **Cloud Cluster:** AWS-hosted MongoDB Atlas with TLS encryption & replica sets
- **Official Driver:** Native MongoDB Node.js Driver v7 (Strictly No Mongoose)
- **Direct Operations:** BSON ObjectIds, native aggregation pipelines, atomic updates
- **Index Optimization:** Unique email, unique orderNumber, and query status indexes

END-TO-END HTTP REQUEST / RESPONSE PIPELINE:

Student/Admin Browser ➔ Express Router ➔ Auth Middleware ➔ Controllers ➔ Services ➔ MongoDB Atlas

Every endpoint responds with uniform JSON shape: { success: true|false, data?: ..., message?: ... }

DATABASE ENGINEERING

Database Design: Collections & Document Schemas

High-performance document storage using the official MongoDB Node.js driver

db.users — User Authentication & Profiles

- **Fields:** _id, name, email, passwordHash, role, createdAt
- **Roles:** 'student' | 'admin' (strictly validated)
- **Security:** Passwords hashed via bcrypt (10 rounds); never stored in plain text
- **Indexes:** Unique index on email (prevents duplicate registrations)

db.menuItems — Food Catalogue & Availability

- **Fields:** _id, name, description, price, category, isAvailable, preparationTime
- **Categories:** Burgers, Pizza, Snacks, Indian Food, Beverages, Desserts
- **Prep Times:** Individual dish cooking durations in minutes (e.g. 5m, 12m)
- **Indexes:** Secondary indexes on category and isAvailable for fast lookups

db.orders — Order Lifecycle & Transactions

- **Fields:** orderNumber, userId, items[], subtotal, tax, campusFee, totalAmount
- **State Machine:** status: 'Placed' | 'Preparing' | 'Ready' | 'Completed' | 'Cancelled'
- **Payment:** paymentMethod ('Cash/UPI at Counter'), paymentStatus ('Pending'|'Paid')
- **Integrity:** Unique index on orderNumber; sparse index on idempotencyKey

db.sessions — MongoDB Session Persistence

- **Engine:** connect-mongo v6 storing Express session state directly in Atlas
- **Fields:** _id (session ID), session (JSON payload), expires (timestamp)
- **Durability:** User authentication persists seamlessly across server reboots
- **TTL Index:** Automatic background expiration of inactive sessions

BACKEND ALGORITHMS & LOGIC

Engineering Beyond the Interface: Backend Integrity

Defensive programming, zero client trust, and mathematical precision

1. Zero-Client-Trust Price Recalculation

Client sends only menuItemId and quantity. The server queries authoritative prices directly from MongoDB, recalculating subtotals to completely prevent client-side tampering.

2. Idempotency-Based Duplicate Prevention

Every checkout submits an idempotencyKey. Rapid double-clicks or browser retries return the existing order without creating duplicate kitchen tickets or double charges.

3. Controlled State Machine & Auto-Settlement

State transitions follow strict rules: Placed ➔ Preparing ➔ Ready ➔ Completed. Transition to Completed automatically marks counter payment as 'Paid'.

4. Smart Preparation Time Estimation Algorithm

Estimated Time = Max(dishPrepTimes) + (totalItems - 1)*2 + (activeQueueOrders)*3. Clamped realistically between 5 and 45 minutes.

ACTUAL CALCULATION EXAMPLE (FROM CART):

Thick Cold Coffee w/ Ice Cream	= ₹85.00
Masala Chai & Ginger Infusion	= ₹30.00
Fiery Jalapeno & Corn Pizza	= ₹169.00
Dal Makhani & Jeera Rice Bowl	= ₹150.00
Subtotal	**₹434.00**
GST Tax (5%)	₹21.70
Campus Convenience Fee	₹10.00
TOTAL PAYABLE	**₹465.70**

✓ **Exactly matches the live cart & checkout screenshots shown on Slide 5.**

RELIABILITY & QUALITY ASSURANCE

Security Architecture & Comprehensive QA Testing

Multi-layered defensive safeguards backed by automated integration & E2E verification

DEFENSE-IN-DEPTH SECURITY ARCHITECTURE

- **Password Hashing:** bcryptjs with 10 salt rounds; plain text passwords never reach DB
- **Session Hardening:** HTTP-only, SameSite cookies preventing Cross-Site Scripting (XSS)
- **Role-Based Access Control:** requireAuth, requireAdmin & protectAdminPages route guards
- **Cross-Student Privacy:** HTTP 403 Forbidden enforced if student accesses other orders
- **Brute-Force Rate Limiting:** loginLimiter restricts login attempts to 10 requests / 15 minutes
- **HTTP Security Headers:** Helmet-configured CSP, X-Content-Type-Options, & Frameguard
- **Server-Side Input Validation:** Strict type, range, and sanitization checks on all payloads

VERIFIED AUTOMATED QA SUITES (100% PASS)

✓ 41 / 41 Integration Tests `[npm test]`

Verifies technology compliance (0 prohibited frameworks), port binding, security headers, auth lifecycle, pricing enforcement, idempotency, privacy locks, and all 8 frontend routes.

✓ 8 / 8 End-to-End Workflows `[npm run test:e2e]`

Validates complete student journeys: ordering, polling tracking, admin status transitions, counter cash settlement, cross-student privacy rejection, price tampering blocks.

✓ 8 / 8 Live Atlas Cloud Metrics `[npm run audit:atlas]`

13 internal deep verification steps: ping, CRUD permissions, indexing, menu seeding, reconnect persistence, student auth, Atlas session storage, aggregation analytics, and graceful offline failover.

Live Demonstration & Architectural Roadmap

Demonstrating end-to-end capabilities today, expanding features tomorrow

LIVE CLASSROOM DEMONSTRATION SEQUENCE (7–10 MINUTES):

1. Student Login: Authenticate as student salik@gmail.com; session cookie issued | **2. Menu Selection:** Filter by 'Beverages' & 'Burgers'; add items to cart | **3. Checkout:** Select 'Cash at Counter', review bill, receive token #CB-XXXXXX | **4. Admin Queue:** Switch to /admin-orders.html; kitchen updates Placed ➔ Preparing | **5. Live Tracking:** Student /orders.html reflects Preparing with countdown via polling | **6. Completion:** Kitchen marks Ready ➔ Completed; payment auto-settles to Paid |

FUTURE ARCHITECTURAL ROADMAP (PLANNED ENHANCEMENTS):

- **Multi-Vendor Stalls:** Extend data model with stallId to support distinct canteen vendor portals (Planned enhancement — not in current baseline)
- **Online Payment Gateway:** Integrate Razorpay or Paytm UPI Intent for direct in-app payments (Planned enhancement — not in current baseline)
- **WebSockets / Server-Sent Events:** Push notifications replacing 4.5s polling for ultra-low latency updates (Planned enhancement — not in current baseline)
- **SMS & WhatsApp Alerts:** Twilio-powered SMS when order transitions to 'Ready' for counter pickup (Planned enhancement — not in current baseline)
- **Predictive Demand Analytics:** Machine learning forecasting ingredient requirements based on weekday schedules (Planned enhancement — not in current baseline)

CSE326 ACADEMIC PROJECT SUMMARY

CampusBite — Making Campus Dining Smarter

✓ **Eliminated Congestion:** Asynchronous digital ordering replaces chaotic physical counter bottlenecks

✓ **Operational Clarity:** Live 4-stage tracking gives students certainty and kitchen staff full queue control

✓ **Strict Compliance:** 100% compliant with CSE326 rules: HTML5, CSS3, Vanilla JS, Node.js, MongoDB Atlas

✓ **Production Reliability:** Validated by 41 integration tests, 8 E2E workflows, and live Atlas persistence

Thank You! — Questions & Answers

Pirzada Salik | Course Code: CSE326 | [INSERT UNIVERSITY NAME]